Final Defense Presentation on

AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics

Presented By

Md. Riaz

Program: B.Sc. in CSE (Diploma)

ID: 0322310105101024

Batch: 16th

8th Semester / 4th Year

Session: Spring - 2023

Supervised By

Mst. Sahela Rahman

Lecturer

Dept. of CSE, PUB

Department of Computer Science & Engineering

Pundra University of Science & Technology, Bogura – 5800

Outline

1. Introduction
2. Problem Statement
3. Literature Review
4. Research Gap and Objectives
5. Methodology
6. AEGIS Architecture
7. Semantic Layer and Compiler
8. Query-to-SQL Walkthrough
9. Prototype Implementation
10. Evaluation Dataset and Results
11. Limitations, Conclusion, and Future Work

Introduction

Motivation:

- E-commerce systems store rich operational data, but built-in dashboards answer only a fixed set of questions.
- When users need new analytical combinations, the request usually depends on a developer or BI person to translate it into report logic.
- Natural language analytics can reduce that delay for managers, sales teams, and administrators.

Central Tension:

- Direct LLM-to-SQL systems are flexible, but the model writes executable database code.
- A safe analytics system should understand the question without giving the model authority to create arbitrary SQL.

AEGIS Direction:

- Use the LLM for intent extraction only.
- Use deterministic semantic mapping and SQL compilation for execution.

Problem Statement

Current Generative NL2SQL approaches have 3 structural vulnerability classes:

1. Structural Injection Risk

Adversarial prompt manipulation can bypass model instructions, causing neural models to output data-modifying DML/DDL statements (DROP, DELETE, UPDATE).

2. Unbounded Schema Hallucination

Probabilistic token generation leads to hallucinated table joins, non-existent entity relations, and invalid column attributes.

3. Access Control & Context Bypass

Direct query generation bypasses application-level multi-tenant boundaries and row-level security scopes.

Literature Review

Work	Main focus	Control approach	Gap for AEGIS
NaLIR [1]	Interactive NLIDB	Parse and refine intent with grammar and DB mapping	Limited dashboard and modern LLM risk handling
nl4dv [2]	NL to visualization specs	Extract analytic attributes; not SQL-centered	Visualization focus, not safe database execution
DashBot [3]	Dashboard generation	Insight-driven dashboard selection	Does not solve arbitrary SQL authority risk
PICARD [4]	Constrained decoding	Parser-level SQL validity during token generation	Model still generates SQL text
G-SQL [5] / TriSQL [6]	Robust Text-to-SQL	Schema-aware generation with rules, repair, and refinement	Safety depends on controlling generated SQL
AEGIS	Safe NL analytics	Intent extraction only; SQL compiled from semantic layer	Trades open SQL for auditable analytics intent validation

Research Gap

Observed Gap in Existing Systems:

- Many systems improve SQL generation accuracy, but still let executable SQL be authored by the model.
- Visualization systems help users express analysis, but do not fully address database execution safety.
- Parser and repair methods can reject invalid syntax, yet syntactically valid SQL can still be unsafe or semantically wrong.

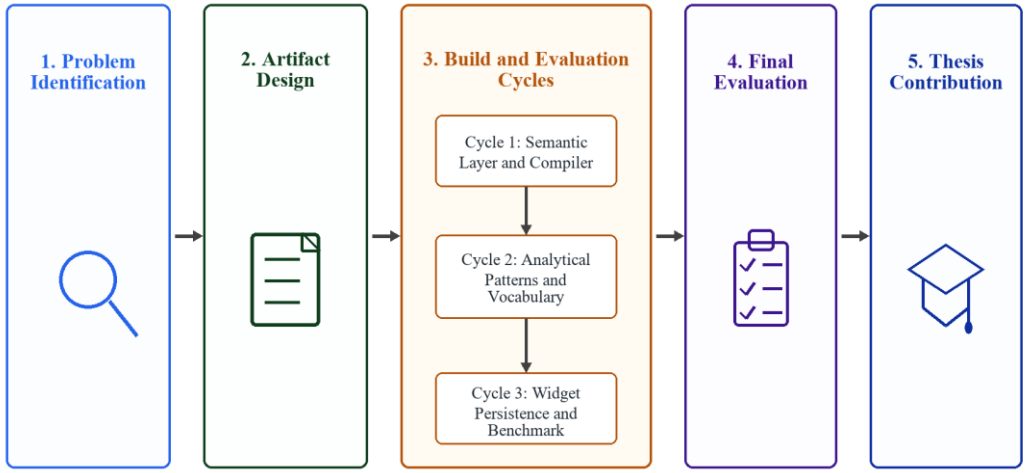
AEGIS Research Gap:

- A practical architecture is needed where natural language is accepted, but executable SQL is produced only from an explicit semantic layer and deterministic compiler.

Objectives and Contributions

Objective	Contribution
Reduce execution risk	Separate natural-language understanding from SQL creation.
Make analytics auditable	Define metrics, dimensions, filters, join paths, and output shapes in a semantic layer.
Support real e-commerce reports	Implement AEGIS over a seeded nopCommerce [7] MySQL dataset and admin analytics oracles.
Evaluate with static datasets	Use fixed natural-language questions, semantic-intent validation tests, and admin-fidelity checks for reproducible evidence.

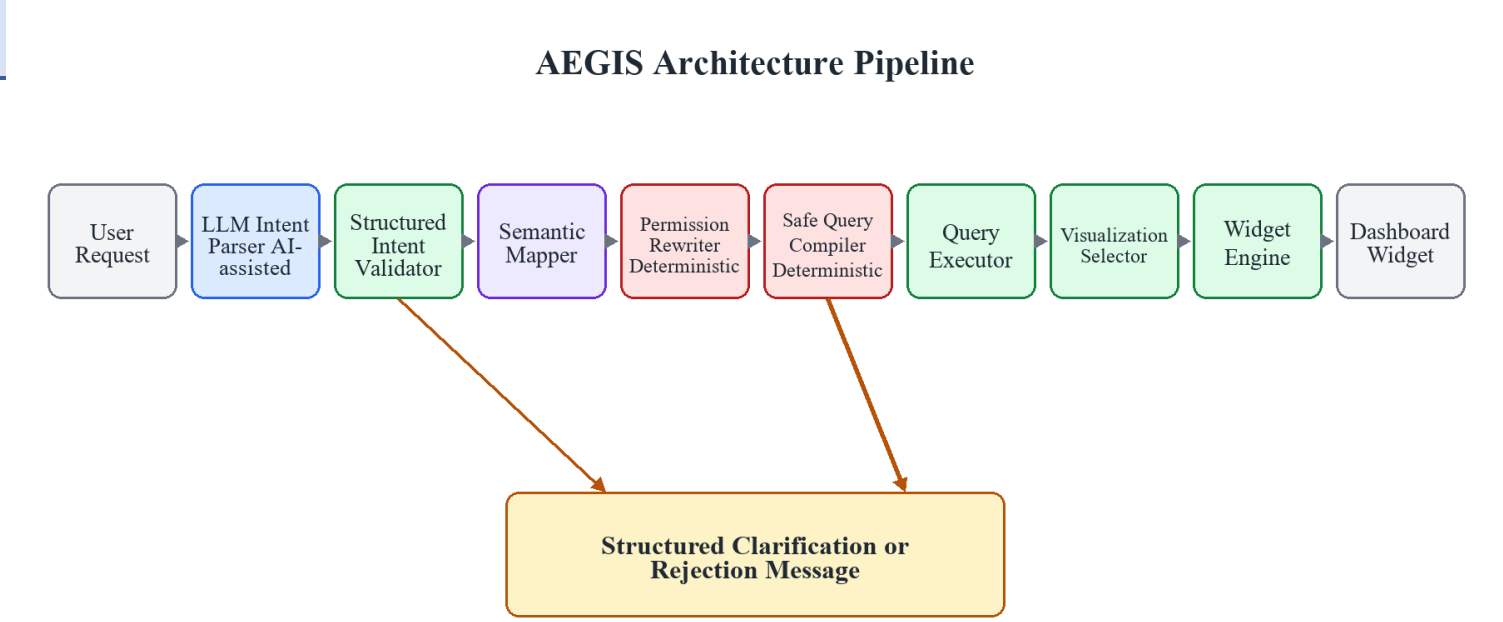
Research Methodology



Design Science Research Process:

- Identify the unsafe execution problem in LLM-assisted database analytics.
- Design a constraint-based architecture where SQL is compiled from explicit definitions.
- Implement the prototype for nopCommerce [7] on MySQL.
- Evaluate with static datasets and executable oracle checks.
- Refine implementation gaps before freezing thesis claims.

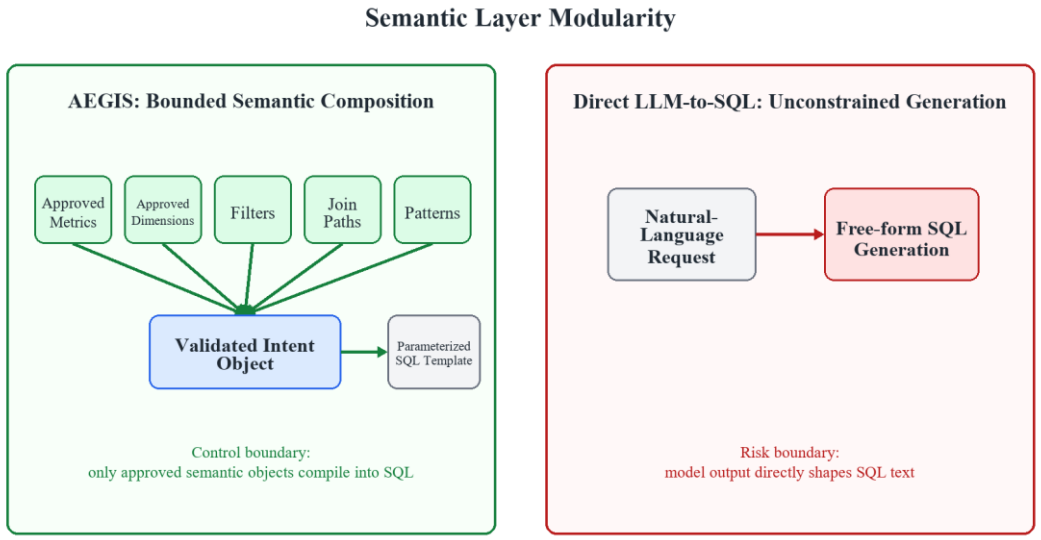
Proposed AEGIS Architecture



Only Stage 1 uses an LLM. Later stages validate structured intent and enforce deterministic compilation, execution, visualization, and persistence.

Main architectural rule: the LLM extracts intent only; approved code maps that intent to semantic definitions and compiled SQL.

Semantic Layer Implementation



Implemented per target system:

- **Business vocabulary:** synonyms for metrics, dimensions, filters, and time periods.
- **Metric definitions:** SQL expressions such as order count, revenue, average order value, and customer count.
- **Dimension definitions:** status, customer, product, manufacturer, category, date, and geography.
- **Join policy:** allowed tables and join paths from the nopCommerce [7] schema.
- **Output contracts:** table, matrix, KPI, and chart-ready shapes.

Intent Extraction Boundary

What the LLM may produce:

- Intent class
- Metric term
- Dimension term
- Time range
- Filter terms
- Requested output shape

What the LLM may not produce:

- Raw SQL
- Table names as executable authority
- Join paths
- Write statements
- Arbitrary expressions

```
{
  "intent_class": "segment",
  "metric_term": "avg_order_value",
  "dimension_term": "order_status",
  "time_range": "all_time",
  "output_shape": "table"
}
```

Boundary claim:

- The model output is treated as data.
- The compiler accepts only fields that map to semantic-layer entries.
- If a required mapping is absent, the system returns a controlled refusal or clarification instead of inventing SQL.

Question-to-SQL Walkthrough

User question
"Show order average by status"

Intent JSON
metric=avg_order_value
dimension=order_status

Semantic mapping
metric + dimension + matrix pattern

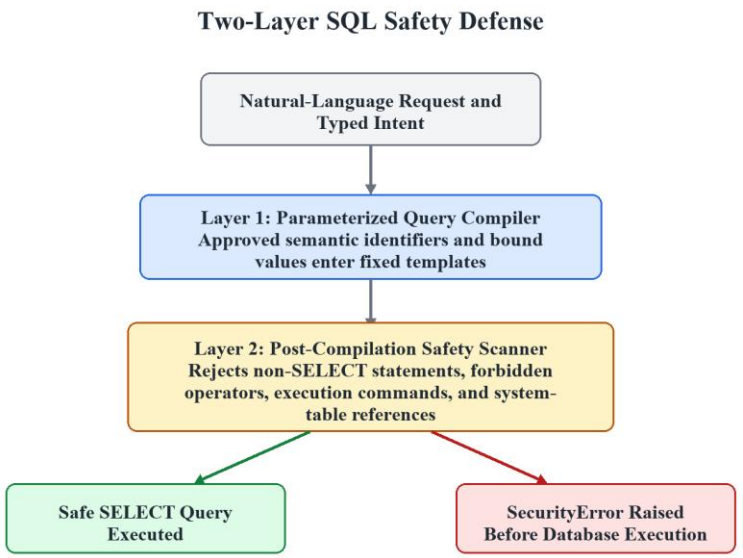
Compiler output
safe SELECT only

```
                SELECT status_label,
       COUNT(*) AS order_count,
       SUM(OrderTotal) AS revenue,
       AVG(OrderTotal) AS avg_order_value
FROM approved_order_summary
GROUP BY status_label
ORDER BY revenue DESC;
```

Status	Orders	Revenue	AOV
Complete	1,846	Tk ...	Tk ...
Processing	312	Tk ...	Tk ...
Pending	226	Tk ...	Tk ...
Cancelled	116	Tk ...	Tk ...

Proof point: the SQL is selected and compiled from known metric/dimension definitions; the model never writes the SELECT statement.

SQL Compiler and Safety Controls



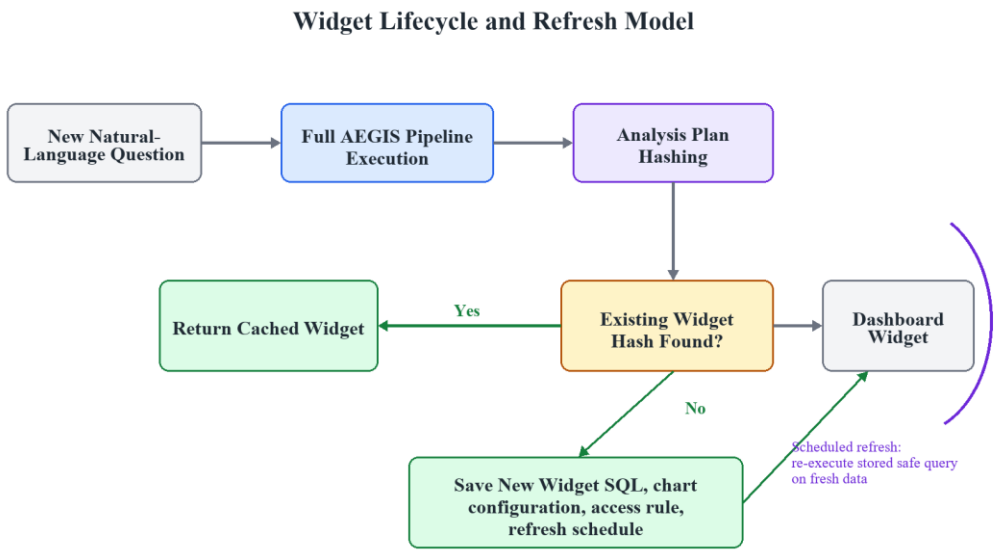
Compiler responsibilities:

- Resolve metric and dimension ids from the semantic layer.
- Add only allowed joins and filters.
- Compile read-only SQL for MySQL.
- Validate SQL structure before execution.

Safety controls:

- No DDL or DML.
- No unapproved tables or columns.
- No silent fallback to a plausible metric.
- Refuse or clarify unsupported requests.

Output Generation



Output is based on result shape, not model narration:

- **KPI:** one numeric value with label.
- **Table:** row/column data with responsive scrolling in the prototype.
- **Matrix:** period columns such as today, week, month, year, and all time.
- **Chart-ready output:** categories and values are produced from query results.

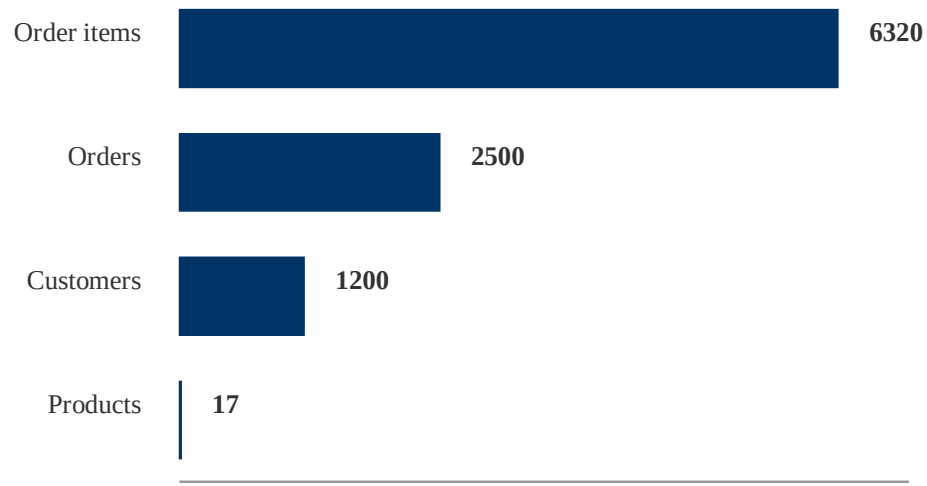
This lets the prototype show both the answer and the evidence path used to produce it.

Prototype Implementation

Prototype stack:

- FastAPI backend for NL analytics requests.
- MySQL seeded with nopCommerce-style [7] commerce data.
- OpenAI-compatible LLM API for intent extraction.
- Semantic-layer files define metrics, dimensions, filters, and output shapes.
- Frontend shows answer output plus AEGIS stage evidence.

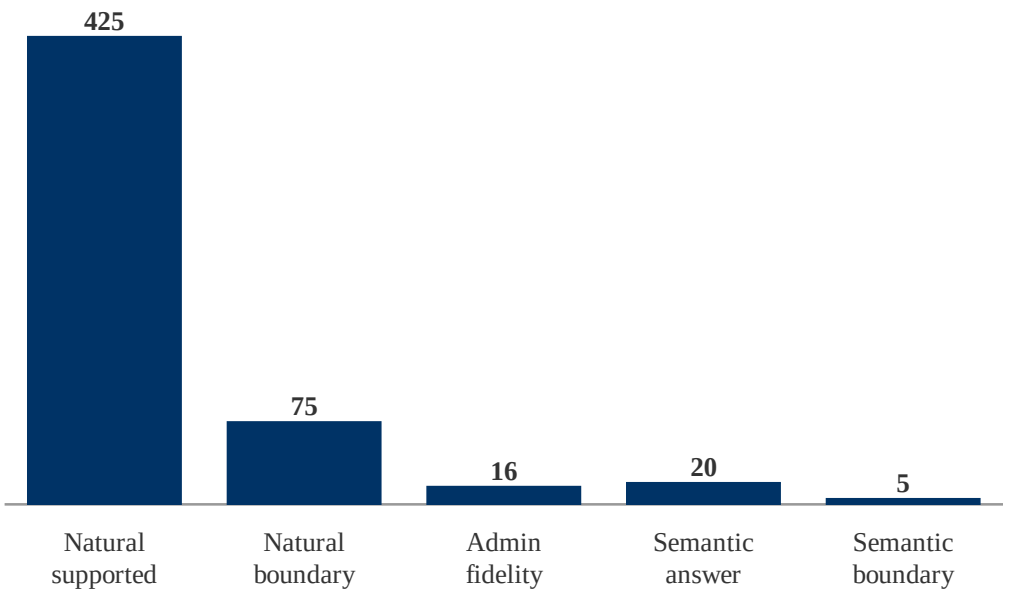
Seeded Data Used in Live Prototype



Reproducibility:

- Docker build includes app, database seed scripts, and smoke checks.
- Local health test verified database connection and seeded counts.

Benchmark Dataset and Results



Dataset / check	Tested	Passed	Meaning
Natural questions	425 supported	425	Answerable scope passes
Natural boundary	75 out-of-scope	75	Boundary labels pass
Admin fidelity	16 reports	16 x3	Execution, shape, result
Semantic intent validation	20 supported + 5 boundary	20 + 5	Vocabulary intent validation passes
Safety	Accepted benchmark SQL	No unsafe SQL	Write/unsupported blocked

Interpretation: the benchmark checks validate the implemented nopCommerce [7] semantic-layer scope.

Beneficiaries and Expected Impact

Beneficiary	Current difficulty	AEGIS impact
Business users	Need reports but need new report combinations	Ask natural-language analytics questions and receive controlled table or chart-ready results
Developers / BI teams	Repeated report requests consume implementation time	Reusable semantic definitions reduce one-off SQL writing for common analytics
Database administrators	Model-written SQL is hard to audit and restrict	Only approved metrics, dimensions, joins, and read-only SQL reach the database
Researchers	NL2SQL work often mixes language accuracy with execution authority	Shows an architecture where language understanding and SQL authority are separated

Expected impact: safer self-service analytics for repeated e-commerce reporting, without giving the LLM permission to author executable SQL.

Scope and Limitations

Current evaluation scope:

- The prototype is implemented and tested for nopCommerce-style [7] analytics on MySQL.
- The semantic layer is intentionally finite and deployment-specific.
- Unsupported structured intents are expected to be rejected or clarified, not guessed.

Prototype limitations:

- Vague unsupported language can still be misread during LLM intent extraction.
- Additional commerce datasets would strengthen external validity.
- Other SQL dialects require compiler extensions.

Not a thesis limitation:

- Multi-turn conversation is outside this thesis because the work evaluates single-request natural-language analytics.

Conclusion

Conclusion:

- AEGIS keeps the LLM useful for language understanding while removing SQL-authoring authority from the model.
- The semantic layer makes analytics definitions explicit, auditable, and reusable for a target system.
- The compiler produces read-only SQL from approved definitions and blocks unsupported execution paths.
- The nopCommerce [7] prototype, static datasets, and admin-fidelity checks show that the approach can be implemented and evaluated practically.

Main contribution:

- A constraint-based architecture for safe LLM-assisted natural language analytics.

Future Work

Possible extensions:

- Evaluate AEGIS on another e-commerce system by replacing only the semantic layer.
- Add more oracle reports from real admin and business workflows.
- Extend compiler modules for PostgreSQL and SQL Server.
- Improve clarification behavior for ambiguous but answerable user questions.
- Add richer chart recommendation while keeping SQL generation deterministic.

References

[1] Li, F., & Jagadish, H. V. (2014). Constructing an interactive natural language interface for relational databases. PVLDB.

[2] Narechania, A., Srinivasan, A., & Stasko, J. (2021). nl4dv: A toolkit for generating analytic specifications from natural language. IEEE TVCG.

[3] Deng, D., Wu, A., Qu, H., & Wu, Y. (2023). DashBot: Insight-driven dashboard generation. IEEE TVCG.

[4] Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: Parsing incrementally for constrained auto-regressive decoding. EMNLP.

[5] Shalaan, H. S. et al. (2025). G-SQL: A schema-aware and rule-guided approach for robust natural language to SQL translation. IEEE Access.

[6] Su, X. et al. (2026). A robust natural language text-to-SQL generation framework with dynamic strategies based on LLMs. Scientific Reports.

[7] nopSolutions. nopCommerce open-source e-commerce platform. GitHub repository.

Thank You & Discussion

THANK YOU!

Questions & Final Defense Discussion